Using KNN as Model Classifier

instead of detecting exact age as final code suggests

In [28]:

```
for i, layer in enumerate(model.layers):
  layer._name = 'layer_' + str(i)
  model.summary()
```

```
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 layer_0 (Functional)        (None, 7, 7, 512)         14714688

 flatten_1 (Flatten)         (None, 25088)             0

 dense_3 (Dense)             (None, 4000)              100356000

 dropout_1 (Dropout)         (None, 4000)              0

 dense_4 (Dense)             (None, 200)               800200

 dense_5 (Dense)             (None, 18)                3618

=================================================================
Total params: 115,874,506
Trainable params: 101,159,818
```

In [33]:

```
# extracting features from second last layer

import pandas as pd
import numpy as np
from PIL import Image
from tensorflow import keras

df = pd.read_csv('/content/wiki_final_file.csv')

path_list = []
feature_list = []
label_list = []

layer_name = 'layer_4'

full_path = df['full_path']
labels = df['true_label']


for i in range(20000): # adjust number according to dataset size or use 'len(full_path)
  the_image = Image.open(full_path[i])
  the_image_array = np.array(the_image)
  new_shape = np.reshape(the_image_array, (1, 224, 224, 3))
  prediction = model.predict(new_shape)
  intermediate_layer_model = keras.Model(inputs=model.input,
                                         outputs=model.get_layer(layer_name).output)

  intermediate_output = intermediate_layer_model(new_shape)
  final_inter_output = np.reshape(intermediate_output,(200))
  path_list.append(full_path[i])
  label_list.append(labels[i])
  feature_list.append(final_inter_output)
  print(i)


```

```
Streaming output truncated to the last 5000 lines.
15000
15001
15002
15003
15004
15005
15006
15007
15008
15009
15010
15011
15012
15013
15014
15015
15016
15017
```

In [34]:

```
# training knn classifier
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt


arr_1 = np.array(feature_list)
arr_2 = np.array(label_list)

neigh = KNeighborsClassifier(n_neighbors=3)
neigh.fit(feature_list, label_list)

```

Out[34]:

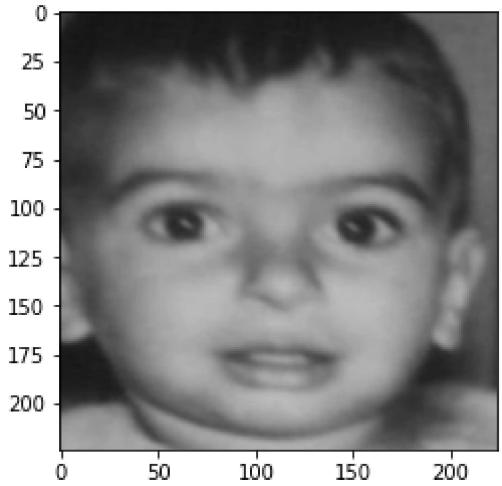
```
KNeighborsClassifier(n_neighbors=3)
```

In [35]:

```
import pandas as pd
import numpy as np
from PIL import Image
from tensorflow import keras

df = pd.read_csv('/content/fgnet_final_file.csv')


full_path = df['full_path']
labels = df['true_label']
```

In [41]:

```
# training knn classifier
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt

base_dir = '/content/cropped_fgnet/'

pred_list = []
test_list = []


loop_num = 0
loop_range = 100

for path in full_path:
  final_dir = base_dir + path
  try:
    the_image = Image.open(final_dir)  # change number her to get diff prediction
  except:
      print(loop_num)
  the_image_array = np.array(the_image)
  new_shape = np.reshape(the_image_array, (1, 224, 224, 3))
  intermediate_output = intermediate_layer_model(new_shape)
  plt.imshow(the_image)
  plt.show()
  print(neigh.predict(intermediate_output))
  the_pred = neigh.predict(intermediate_output)
  pred_list.append(the_pred)
  test_list.append(labels[loop_num])
  loop_num += 1
  if loop_num > loop_range:
    break

```




```
['[0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]']
```

In [37]:

```
from sklearn import metrics
# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(test_list, pred_list))
```

```
Accuracy: 0.12871287128712872
```

In [38]:

```
print(neigh.predict(intermediate_output))
```

```
['[0. 0. 0. 0. 1. 0. 0. 0. 0. 1.]']
```